

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені І.І. МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

ЛАБОРАТОРНА РОБОТА №4

з дисципліни
«Інтелектуальний аналіз даних»

на тему:

Linear Discriminant Analysis (LDA)

студента 4 курсу
спеціальності: Інформаційні системи і технології

Краковський Владислав Олександрович

(Прізвище, ім'я та по батькові)

Викладач: доц. Петрушина Т.І.

(Прізвище, ім'я та по батькові)

Одеса – 2026

1. Постановка задачі:

Ознайомитися з методом лінійного дискримінантного аналізу (LDA), навчитися будувати модель класифікації на основі багатовимірних даних, реалізувати метод у середовищах Microsoft Excel та Python, а також провести класифікацію невідомих об'єктів на основі отриманої моделі.

2. Хід роботи:

Linear Discriminant Analysis (LDA) — це метод контрольованого машинного навчання, який використовується для класифікації об'єктів.

На відміну від:

- K-means clustering — неконтрольованого навчання,
- лінійної регресії — прогнозування числових значень,

LDA виконує саме класифікацію об'єктів за класами.

Метод намагається знайти таку лінійну функцію:

$$P = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + \dots + w_n * x_n + b$$

де:

- x_i — ознаки об'єкта,
- w_i — вагові коефіцієнти,
- b — вільний член,
- P — числовий прогноз.

Після отримання числового значення P воно порівнюється з пороговими значеннями (cutoff), після чого об'єкт відноситься до певного класу.

LDA прагне:

максимально збільшити відстань між класами;

мінімізувати розкид значень всередині кожного класу.

Для цього використовуються:

Міжгрупова дисперсія

Відображає, наскільки далеко розташовані середні значення класів.

Внутрішньогрупова дисперсія

Показує, наскільки сильно об'єкти розкидані всередині класів.

Критерій оптимізації

Чим більше це значення:

тим краще розділені класи;

тим якісніше працює модель.

Підготовка даних

Для роботи використовувався датасет Wine Dataset.

Кожен запис містить:

13 хімічних характеристик вина;

клас виробника:

V1,

V2,

V3.

Було використано:

178 відомих зразків для навчання;

3 невідомих зразки для класифікації.

W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK
	y	Numerical Prediction	Type classification	Difference	Within-g-variances									
	0	0,396585124	V1	0	0,050422775					0	V1			
	0	0,459753248	V1	0	0,026044162					1	V2			
	0	0,618957066	V1	0	4,74464E-06					2	V3			
	0	0,503751262	V1	0	0,013779009							Obtained-by-Solver	Obtained-by-LINEST	
	0	0,94954543	V2	1	0,107853223				b		14	2,7167764	3,473285306	
	0	0,440592181	V1	0	0,032595813				w1		13	-0,063479035	-0,117003844	
	0	0,43957786	V1	0	0,032963099				w2		12	0,032293021	0,030170995	
	0	0,505431675	V1	0	0,013387326				w3		11	-0,043382001	-0,148552157	
	0	0,545650207	V1	0	0,005697997				w4		10	0,026144996	0,039854337	
	0	0,63094818	V1	0	9,62929E-05				w5		9	-0,000389206	-0,000489766	
	0	0,38022708	V1	0	0,058036764				w6		8	0,109629458	0,144320132	
	0	0,624730185	V1	0	1,29233E-05				w7		7	-0,300152298	-0,372391406	
	0	0,583220864	V1	0	0,001437503				w8		6	-0,281546387	-0,303474312	
	0	0,239350364	V1	0	0,145759727				w9		5	0,020936265	0,039356529	
	0	0,269604421	V1	0	0,123573949				w10		4	0,065558695	0,075623866	
	0	0,674792594	V1	0	0,002879107				w11		3	-0,159894064	-0,149245059	
	0	0,655009857	V1	0	0,001147487				w12		2	-0,205413421	-0,2700542	
	0	0,69409411	V1	0	0,00532299				w13		1	-0,000451613	-0,000701139	
	0	0,315865427	V1	0	0,093189687									
	0	0,674706843	V1	0	0,002869912				mean	sample number	cutoff			
	0	0,582848265	V1	0	0,001465896					0,621135286	59	0,924283993		
	0	0,921774395	V1	0	0,090383874					1,176196299	71	1,507714352		
	0	0,362485526	V1	0	0,066899698					1,998084804	48			
	0	0,669031132	V1	0	0,002294012									
	0	0,625847797	V1	0	2,22078E-05									
	0	0,833894892	V1	0	0,04526665									
	0	0,526126136	V1	0	0,009026738									
	0	0,747661368	V1	0	0,01600885									
	0	0,58992207	V1	0	0,000974265									
	0	0,662531449	V1	0	0,001713642									
	0	0,693460231	V1	0	0,005230898									
	0	0,601294966	V1	0	0,000393638									
	0	0,613644855	V1	0	5,61066E-05									
	0	0,60268092	V1	0	0,000340564									
	0	0,720529166	V1	0	0,009879143									
	0	0,719106243	V1	0	0,009598308									
	0	0,738157589	V1	0	0,013694219									
	0	0,836146541	V1	0	0,04622984									
	0	0,672790689	V1	0	0,002668281									
	0	0,656732033	V1	0	0,001267128									
	0	0,696763392	V1	0	0,00571961									
	0	0,82463489	V1	0	0,041412089									
	0	0,403107735	V1	0	0,047033655									

Перетворення класів у числові значення

Для роботи методу LDA категоріальні значення класів були перетворені у числові:

Клас Значення

с я

V1 0

V2 1

V3 2

Для цього використовувалась формула:

=ЕСЛИ(A2=\$AG\$1; AG\$1; ЕСЛИ(A2=\$AG\$2; AG\$2; AG\$3))

де:

A2 — клас поточного об'єкта;

AG1:AG3 — числові значення класів

Отримання коефіцієнтів лінійної функції

Для знаходження коефіцієнтів використовувалась функція:

=ИНДЕКС(ЛИНЕЙН(\$X\$2:\$X\$179;\$B\$2:\$N\$179; ИСТИНА; ИСТИНА); 1; АН5)

де:

\$X\$2:\$X\$179 — числові значення класів;

\$B\$2:\$N\$179 — ознаки C1–C13;

ЛИНЕЙН() — знаходить коефіцієнти лінійної моделі;

ИНДЕКС() — вибирає окремий коефіцієнт.

Обчислення Numerical Prediction

Для кожного об'єкта обчислювалося прогнозоване значення:

=B2*\$AI\$6+C2*\$AI\$7+D2*\$AI\$8+E2*\$AI\$9+F2*\$AI\$10+G2*\$AI\$11
+H2*\$AI\$12+I2*\$AI\$13+J2*\$AI\$14+K2*\$AI\$15+L2*\$AI\$16+M2*\$AI\$17+N2*\$AI\$18+\$AI\$5

де:

B2:N2 — значення ознак поточного об'єкта;

AI6:AI18 — коефіцієнти $w_1...w_{13}$

AI5 — вільний член b .

Результатом є числове значення:

Numerical Prediction.

Обчислення середніх значень класів

Для кожного класу обчислювалося середнє значення прогнозів:
$$=CPЗНАЧЕСЛИМН(\$Y\$2:\$Y\$179;\$X\$2:\$X\$179;AG1)$$

де:

$Y2:Y179$ — Numerical Prediction;

$X2:X179$ — числові класи;

$AG1$ — поточний клас.

Формула обчислює:

середнє значення прогнозів всередині класу.

Підрахунок кількості елементів класу

Використовувалась формула:
$$=СЧЁТЕСЛИ(\$X\$2:\$X\$179;AG1)$$

де:

$X2:X179$ — класи;

$AG1$ — поточний клас.

Формула визначає:

кількість елементів у кожному класі.

Обчислення cutoff

Cutoff використовувався для розділення класів.

Формула:

$$=(AG21*AH21+AG22*AH22)/(AH21+AH22)$$

де:

$AG21, AG22$ — середні значення двох сусідніх класів;

$AH21, AH22$ — кількість елементів цих класів.

Формула знаходить:

порогове значення між класами.

Класифікація об'єктів

Для визначення класу використовувалась формула:

=ЕСЛИ(Y2<\$AI\$21; \$АН\$1; ЕСЛИ(Y2<\$AI\$22; \$АН\$2; \$АН\$3))

де:

Y2 — Numerical Prediction;

AI21, AI22 — cutoff;

АН1 : АН3 — назви класів.

Логіка:

якщо значення менше першого cutoff → V1;

якщо менше другого cutoff → V2;

інакше → V3.

Перевірка правильності класифікації

Використовувалась формула:

=ЕСЛИ(Z2=A2; 0; 1)

де:

Z2 — прогнозований клас;

A2 — реальний клас.

Формула:

повертає 0 при правильній класифікації;

повертає 1 при помилці.

Обчислення внутрішньогрупової дисперсії

Для кожного об'єкта:

=ЕСЛИ(X2=AG\$1; (Y2 - AG\$21) ^2; ЕСЛИ(X2=AG\$2; (Y2 - AG\$22) ^2; (Y2 - AG\$23) ^2))

де:

Y2 — прогноз;

AG21 : AG23 — середні значення класів.

Формула обчислює:

квадрат відхилення прогнозу від середнього значення свого класу.

Загальна внутрішньогрупова дисперсія

=СУММ(AB2:AB179)

де:

AB2:AB179 — відхилення всіх об'єктів.

Міжгрупова дисперсія

$$= (AG21 - \text{CPЗНАЧ}(AG21:AG23))^2 + (AG22 - \text{CPЗНАЧ}(AG21:AG23))^2 + (AG23 - \text{CPЗНАЧ}(AG21:AG23))^2$$

Формула показує:

наскільки далеко розташовані середні значення класів.

Класифікація невідомих об'єктів

Для рядків 182–184 використовувались ті самі формули:

Numerical Prediction;

cutoff;

Type classification.

У результаті було отримано:

Рядо	Кла
------	-----

к	с
---	---

182	V2
-----	----

183	V2
-----	----

184	V1
-----	----

Python реалізація:

```
[7]: # Лабораторная работа 4: Linear Discriminant Analysis в Python

#Задача: по химическим признакам вина `C1-C13` определить производителя `V1`, `V2`, `V3`.

#В ноутбуке реализованы два варианта:

#1. **LINEST-аналог**, как в Excel через линейную модель.
#2. **Solver-аналог**, где коэффициенты оптимизируются по отношению `Inter-group variance / Within
```

```
[8]: import pandas as pd
import numpy as np

file_path = "Chapter4-HW.xlsx"

raw = pd.read_excel(file_path, sheet_name="LDA")

# Берем только исходные данные: Vendor + C1:C13
data = raw.iloc[:, :14].copy()

# Обучающие строки: известные производители V1, V2, V3
train = data[data["Vendor"].isin(["V1", "V2", "V3"])].copy()

# Строки для классификации: Vendor = ?
unknown = data[data["Vendor"].eq("?")].copy()

features = [f"C{i}" for i in range(1, 14)]

train.head()
```

```
[8]:
```

	Vendor	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13
0	V1	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.0
1	V1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.0
2	V1	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185.0
3	V1	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480.0
4	V1	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735.0

[9]: *# Кодируем классы как в Excel:*

```
# V1 -> 0  
# V2 -> 1  
# V3 -> 2
```

```
class_to_num = {  
    "V1": 0,  
    "V2": 1,  
    "V3": 2  
}
```

```
num_to_class = {  
    0: "V1",  
    1: "V2",  
    2: "V3"  
}
```

```
X = train[features].astype(float).values  
y = train["Vendor"].map(class_to_num).astype(float).values
```

```
# Добавляем столбец единиц для свободного коэффициента b  
X_design = np.column_stack([np.ones(len(X)), X])
```

```
print("Размер обучающей выборки:", X.shape)  
print("Количество объектов по классам:")  
print(train["Vendor"].value_counts().sort_index())
```

Размер обучающей выборки: (178, 13)

Количество объектов по классам:

Vendor

V1 59

V2 71

V3 48

Name: count, dtype: int64

```
[10]: ## 1. LINEST-аналог

#В Excel использовалась функция `ЛИНЕЙН`.

#В Python то же самое можно сделать методом наименьших квадратов:

#\[
#y = b + w_1x_1 + w_2x_2 + ... + w_{13}x_{13}
#\]
```

```
[11]: # Аналог Excel LINEST
linest_coefficients = np.linalg.lstsq(X_design, y, rcond=None)[0]

coef_names = ["b"] + [f"w{i}" for i in range(1, 14)]

linest_table = pd.DataFrame({
    "coefficient": coef_names,
    "value": linest_coefficients
})

linest_table
```

```
[11]:
```

	coefficient	value
0	b	3.473285
1	w1	-0.117004
2	w2	0.030171
3	w3	-0.148552
4	w4	0.039854
5	w5	-0.000490
6	w6	0.144320
7	w7	-0.372391
8	w8	-0.303474
9	w9	0.039357
10	w10	0.075624
11	w11	-0.149245
12	w12	-0.270054
13	w13	-0.000701

```

def calculate_predictions(X_design, coefficients):
    return X_design @ coefficients

def calculate_means_counts_cutoffs(predictions, y):
    result_df = pd.DataFrame({
        "y": y,
        "prediction": predictions
    })

    means = result_df.groupby("y")["prediction"].mean()
    counts = result_df.groupby("y")["prediction"].count()

    cutoff_01 = (means.loc[0] * counts.loc[0] + means.loc[1] * counts.loc[1]) / (counts.loc[0] + counts.loc[1])
    cutoff_12 = (means.loc[1] * counts.loc[1] + means.loc[2] * counts.loc[2]) / (counts.loc[1] + counts.loc[2])

    return means, counts, [cutoff_01, cutoff_12]

def classify_by_cutoffs(predictions, cutoffs):
    classes = []

    for value in predictions:
        if value < cutoffs[0]:
            classes.append("V1")
        elif value < cutoffs[1]:
            classes.append("V2")
        else:
            classes.append("V3")

    return np.array(classes)

def count_errors(real_classes, predicted_classes):
    return (real_classes.values != predicted_classes).sum()

def lda_metrics(coefficients, X_design, y):
    predictions = calculate_predictions(X_design, coefficients)
    means, counts, cutoffs = calculate_means_counts_cutoffs(predictions, y)

    inter_group = ((means - means.mean()) ** 2).sum()

    within_group = 0
    for class_number in [0, 1, 2]:
        class_predictions = predictions[y == class_number]
        within_group += ((class_predictions - means.loc[class_number]) ** 2).sum()

    ratio = inter_group / within_group

    return {
        "predictions": predictions,
        "means": means,
        "counts": counts,
        "cutoffs": cutoffs,
        "inter_group": inter_group,
        "within_group": within_group,
        "ratio": ratio
    }

```

```
[13]: linest_metrics = lda_metrics(linest_coefficients, X_design, y)

linest_train_predictions = linest_metrics["predictions"]
linest_cutoffs = linest_metrics["cutoffs"]
linest_predicted_classes = classify_by_cutoffs(linest_train_predictions, linest_cutoffs)

linest_difference = count_errors(train["Vendor"], linest_predicted_classes)

print("LINEST means:")
print(linest_metrics["means"])

print("\nLINEST sample numbers:")
print(linest_metrics["counts"])

print("\nLINEST cutoffs:")
print(linest_cutoffs)

print("\nLINEST Difference =", linest_difference)
print("Inter-group variance =", linest_metrics["inter_group"])
print("Within-group variance =", linest_metrics["within_group"])
print("Inter/within ratio =", linest_metrics["ratio"])

LINEST means:
y
0.0    0.098456
1.0    0.985982
2.0    1.899716
Name: prediction, dtype: float64

LINEST sample numbers:
y
0.0     59
1.0     71
2.0     48
Name: prediction, dtype: int64

LINEST cutoffs:
[np.float64(0.5831819188947683), np.float64(1.354546844431928)]

LINEST Difference = 13
Inter-group variance = 1.6223819580279275
Within-group variance = 9.553963755415872
Inter/within ratio = 0.16981244638993373
```

```
[14]: # Классификация неизвестных строк 182-184 через LINEST
```

```
X_unknown = unknown[features].astype(float).values
X_unknown_design = np.column_stack([np.ones(len(X_unknown)), X_unknown])

unknown_predictions_linest = calculate_predictions(X_unknown_design, linest_coefficients)
unknown_classes_linest = classify_by_cutoffs(unknown_predictions_linest, linest_cutoffs)

unknown_result_linest = unknown[["Vendor"] + features].copy()
unknown_result_linest["Numerical prediction"] = unknown_predictions_linest
unknown_result_linest["Predicted vendor"] = unknown_classes_linest

unknown_result_linest
```

```
[14]:
```

	Vendor	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	Numerical prediction	Predicted vendor
180	?	14.0	4.0	3.0	19.0	97.0	3.0	1.0	1.0	2.0	5.0	1.0	3.0	910.0	0.836467	V2
181	?	13.0	1.0	2.0	14.0	105.0	3.0	1.0	1.0	1.0	7.0	1.0	2.0	1660.0	0.664412	V2
182	?	12.0	1.0	2.0	28.0	82.0	1.0	4.0	1.0	2.0	7.0	1.0	3.0	489.0	0.535162	V1

```
[15]: ## 2. Solver-аналог

#Теперь оптимизируем коэффициенты так, как Solver в Excel: максимизируем отношение

#\[
#\frac{S_G}{S_W}
#\]

#где:

#- \((S_G)\) – межгрупповая дисперсия;
#- \((S_W)\) – внутригрупповая дисперсия.

#В Excel это делалось через Solver. В Python используем `scipy.optimize`.
```

```
[16]: from scipy.optimize import minimize

def objective(coefficients):
    metrics = lda_metrics(coefficients, X_design, y)
    return -metrics["ratio"] # maximize ratio -> minimize negative ratio

# Стартуем с коэффициентов LINEST, как в Excel
start_coefficients = linest_coefficients.copy()

solver_result = minimize(
    objective,
    start_coefficients,
    method="BFGS",
    options={
        "maxiter": 10000,
        "gtol": 1e-9
    }
)

solver_coefficients = solver_result.x

solver_table = pd.DataFrame([
    "coefficient": coef_names,
    "LINEST": linest_coefficients,
    "Solver-like": solver_coefficients
])

solver_table
```

```
[16]:
```

	coefficient	LINEST	Solver-like
0	b	3.473285	3.473285
1	w1	-0.117004	-0.078947
2	w2	0.030171	0.040162
3	w3	-0.148552	-0.053954
4	w4	0.039854	0.032516
5	w5	-0.000490	-0.000484
6	w6	0.144320	0.136342
7	w7	-0.372391	-0.373290

```
[17]: solver_metrics = lda_metrics(solver_coefficients, X_design, y)

solver_train_predictions = solver_metrics["predictions"]
solver_cutoffs = solver_metrics["cutoffs"]
solver_predicted_classes = classify_by_cutoffs(solver_train_predictions, solver_cutoffs)

solver_difference = count_errors(train["Vendor"], solver_predicted_classes)

print("Solver-like means:")
print(solver_metrics["means"])

print("\nSolver-like sample numbers:")
print(solver_metrics["counts"])

print("\nSolver-like cutoffs:")
print(solver_cutoffs)

print("\nSolver-like Difference =", solver_difference)
print("Inter-group variance =", solver_metrics["inter_group"])
print("Within-group variance =", solver_metrics["within_group"])
print("Inter/within ratio =", solver_metrics["ratio"])
```

Solver-like means:

```
y
0.0    0.866994
1.0    1.557312
2.0    2.579469
Name: prediction, dtype: float64
```

Solver-like sample numbers:

```
y
0.0    59
1.0    71
2.0    48
Name: prediction, dtype: int64
```

Solver-like cutoffs:

```
[np.float64(1.2440137134453668), np.float64(1.96961062940652)]
```

```
Solver-like Difference = 14
Inter-group variance = 1.4846387968948056
Within-group variance = 8.588330316932852
Inter/within ratio = 0.17286698835601075
```

```
[18]: # Классификация неизвестных строк 182-184 через Solver-Like коэффициенты

unknown_predictions_solver = calculate_predictions(X_unknown_design, solver_coefficients)
unknown_classes_solver = classify_by_cutoffs(unknown_predictions_solver, solver_cutoffs)

unknown_result_solver = unknown[["Vendor"] + features].copy()
unknown_result_solver["Numerical prediction"] = unknown_predictions_solver
unknown_result_solver["Predicted vendor"] = unknown_classes_solver

unknown_result_solver
```

```
[18]:
```

	Vendor	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	Numerical prediction	Predicted vendor
180	?	14.0	4.0	3.0	19.0	97.0	3.0	1.0	1.0	2.0	5.0	1.0	3.0	910.0	1.606629	V2
181	?	13.0	1.0	2.0	14.0	105.0	3.0	1.0	1.0	1.0	7.0	1.0	2.0	1660.0	1.423838	V2
182	?	12.0	1.0	2.0	28.0	82.0	1.0	4.0	1.0	2.0	7.0	1.0	3.0	489.0	1.004868	V1

```
[19]: ## 3. Сравнение итогов

#Главный результат задания – классификация неизвестных вин в строках 182–184.
```

```
[20]: comparison = pd.DataFrame({
    "Row": unknown.index + 2, # Excel rows
    "LINEST prediction": unknown_predictions_linest,
    "LINEST vendor": unknown_classes_linest,
    "Solver prediction": unknown_predictions_solver,
    "Solver vendor": unknown_classes_solver
})

comparison
```

```
[20]:
```

	Row	LINEST prediction	LINEST vendor	Solver prediction	Solver vendor
0	182	0.836467	V2	1.606629	V2
1	183	0.664412	V2	1.423838	V2
2	184	0.535162	V1	1.004868	V1

```
[21]: # Сохраняем результат в Excel

output_file = "Chapter4_HW_LDA_Python_Result.xlsx"

with pd.ExcelWriter(output_file) as writer:
    linest_table.to_excel(writer, sheet_name="LINEST coefficients", index=False)
    solver_table.to_excel(writer, sheet_name="Solver coefficients", index=False)
    comparison.to_excel(writer, sheet_name="Unknown classification", index=False)

print(f"Результат сохранен в файл: {output_file}")

Результат сохранен в файл: Chapter4_HW_LDA_Python_Result.xlsx
```

Результат прогнозування у Python збігся з результатом, який був отриманий у Excel.

Висновки щодо роботи

У ході лабораторної роботи було досліджено метод Linear Discriminant Analysis.

Було:

- побудовано модель класифікації в Excel;
- реалізовано обчислення коефіцієнтів через LINEST;
- виконано класифікацію невідомих об'єктів;
- досліджено оптимізацію моделі через Solver;
- реалізовано аналогічний алгоритм у Python.

У результаті було встановлено, що метод LDA є ефективним інструментом контрольованої класифікації багатовимірних даних.